

UML è linguaggio unificato

La sono molti compromessi:

• Modellazione di oggetti in UML

L'ANALISI

Input: requisiti

Output: schema concettuale

Obiettivi: costruire un modello pratico completo e semplice

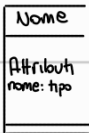
• concentrarsi sul "cosa" e non sul "come"

Serve a ragionare sui requisiti e prendere decisioni ad alto livello sulla strutturazione dell'applicazione

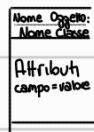
È un linguaggio fatto di diagrammi grafici:

- strutturali (Class & object)
- comportamentali (statechart, cenni)
- architeturali

classi (solo concettuali)



oggetti

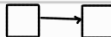


Gli oggetti hanno valori con identità, due oggetti con stessi valori sono oggetti distinti

Le associazioni legano le classi tra di loro (o relazioni)

• Tra più classi ci possono essere più associazioni

• Alcune associazioni hanno un verso (di pertinenza)



• Le associazioni hanno molteplicità che hanno importanza

per la traduzione (contorno delle cardinalità del modello ER)

1 ha min x e max y
2 ha min w e max z
0..* * qualsiasi
1..1 1

i numeri precisi sono rari

- 0..* (qualsunque)
- 1..* (almeno 1)
- 0..1 (possibile evento)
- 1..1

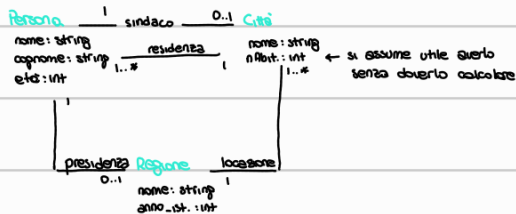
← non specificato
→ non specificato in entrambi i versi

ES 1

persone (nome, cognome, età)

città residenza (nome, abitanti, sindaco)

regione (nome, anno istit., presidente)



Nomi di ruoli



fondamentale sulle associazioni con sé stessi

Anche le associazioni possono avere attributi



associazione con attributi
utili per associazioni molti a molti (* - *)

Una coppia di oggetti o esiste o meno, non posso avere più coppie degli stessi oggetti o non sarebbe più un'associazione => devo promuoverlo a oggetto

Associazioni tra più oggetti vengono definite N-arie (con N numero di oggetti) ← usale solo se non divisibili in 3 classi

part-of:

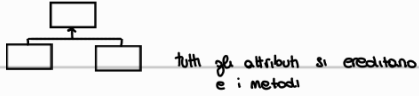
simbolo con il rombo della parte racchiudente



Composizione

port-of più forte: l'oggetto senza quello contenente non ha senso. Es capitolo non ha senso senza libro

15-A (Gerarchia)



Operazioni e Metodi

NomeMetodo (parametri) si dividono in interrogazioni (selettori) e modifiche
 si mettono solo i metodi importanti

dichiarazione NomeOperazione (NomeParametro: Tipo, ...): TipoRisultato (se omissa è void)

Le postcondizioni (o parole) descrivono cosa fa l'operazione; le precondizioni descrivono le condizioni che i dati devono rispettare (le descrivo a parole)

I tipi possono essere velt, insieme, tipi primitivi o classi definite nell'UML

ES biblioteca

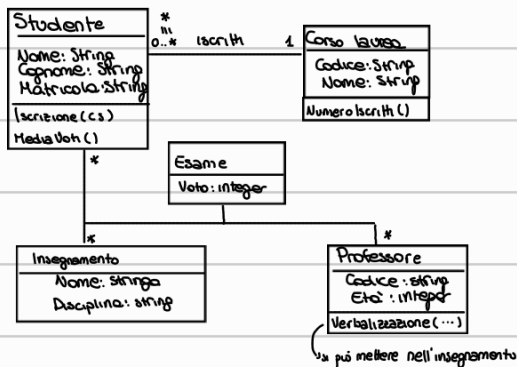
Acquisisce (l: libro): il libro l viene acquisito dalla biblioteca
 Precondizioni: la biblioteca non possiede il libro l

Prestito (l: libro, p: persona): l viene dato a p
 Pre: biblio possiede l e non è in prestito

Restituzione (l: libro): il libro l viene restituito
 pre: l è in prestito

InPrestito (l: libro): boolean: Restituisce true se il libro è in prestito

Prestiti (p: persona): vettore(libro): restituisce il vettore di libri prestati a p



Le relazioni ternarie sono praticamente sempre M a M a M

Diagrammi comportamentali ← come si evolvono

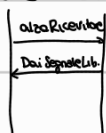
Diagrammi di casi d'uso → spiega le interazioni tra utenti e sistema



Diagrammi di sequenza

Esempi di interazione tra oggetti

Utente Centrale



• Activity Diagram

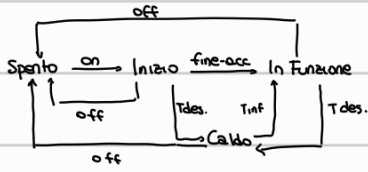
Enfatti 23a quando un'operazione viene fatta con quali operazioni aspettare prima di andare in esecuzione

• Diagrammi di Stato

o Stato → valore di un insieme di attributi specifici

o Eventi → Azione che provoca una transizione di stato

Gli stati e gli eventi si riferiscono ad un **unico** oggetto



Esiste il concetto di **stato iniziale** e **stato finale**; posso avere molteplici



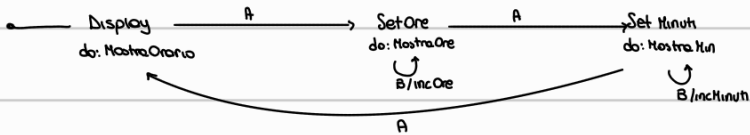
Si possono aggiungere delle condizioni. es. Spento **on[acqua non gelata]** Inizio

hanno una certa durata ← eseguita negli stati

Esistono **azioni** e **attività** do: attività

eseguito nell'evento
evento [condizione]/azione

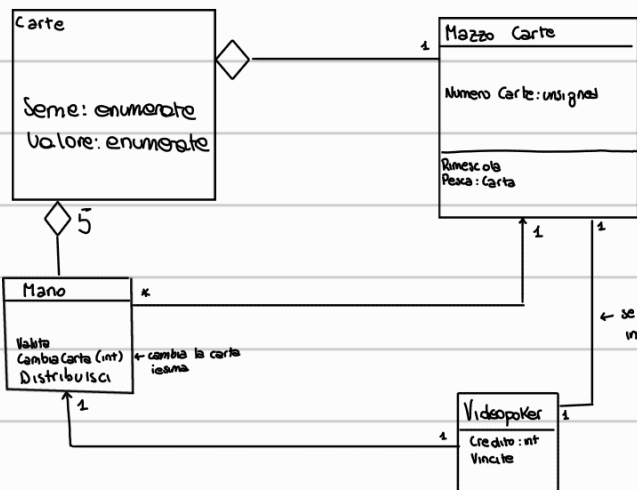
Es. orologio : Display a cristalli. Tasto A cambia selezione B incrementa di 1



Gli stati possono essere anonimi e contengono un'operazione che al termine passa allo stato successivo

ciao, come stai?

Poker online



Mazzo d(3); // mazzo con 3 pacchetti ← c'è anche costruttore
 Carte c;
 c = d.Pesca(); fornisce la carta in cima

Mazzo d;
 Mano m(&d); // m2(&d);
 m.Distribuisce();
 cout << d.NumCarte(); stampa 47
 // m2.Distribuisce(); m2 avrà carte diverse da m1

← se non lo mette non avrà modo di rimiscolare il mazzo ma dovrà
 inserire un metodo nella mano per rimiscolare il proprio mazzo

in c non si mette

Enum class Nome di valore 1, ..., valore N;

class carte

enum class Seme { Cuori, quadri, fiori, picche };

enum class Valori { asso, re, donna, fante, dieci, ..., tre, due }; // senso decrescente

Si dovrà fare vector<Carte> m; perché la carta non dipende dal mazzo

Posso giocare con numero semi o valori diversi (<max) i valori sono messi come costanti pubbliche assegnati dal costruttore

Il mazzo ha solo un valore vector<Carte> m;

Per mescolare il mazzo a via il Fisher-Yates shuffle nel 1928

Tiro un numero a caso tra 0 e N-1 e lo swap con la prima posizione

Per tra i e N-1 e lo scambio con il secondo

Per rimiscolare faccio reverse con tutte le carte (anche se ho più mazzi)
 Sì, riorcina il mazzo con tutte le carte a seconda del numero di mazzi

mazzo[k] = Carta(i / Num.Valori, i % Num.Valori)

```

Mescolare: for(int i=0; i<mazzo.size()-1; i++)
{
    j = Random(i, mazzo.size()-1);
    swap(mazzo[i], mazzo[j]);
}
  
```

driver mazzo → crea un mazzo con N pacchetti
 → pesca 1 carta
 → vedo quante ne rimangono
 → rimiscola } funzioni per provare le funzioni del mazzo

Il make fa il primo ma se aggiungo nella prima posizione all: toth_gli_exe per forzarlo a fare tutti i make e non solo il primo

Mano

è un vettore di carte (interne alla mano; non ho accessi esterni)
 5 elementi

Per valutare i punti ho un enum ordinato in base ai punti

Mano(Mazzo* m); ← dal costruttore e non ho modo per di modificarlo

Punto Valuta() const; ← complicato

Mano:: Mano (Mazzo * m)

: carte (s) ← il costruttore crea un mazzo di 5 assi di cuori ← costruttore default con asso di cuori

```
if (m → NUM_MAZZI != 1)
    throw logic_error;
mazzo_utilizzato = m;
// Eventuale distribuzione per consistenza ← mai mano inconsistente
```

Mano:: Distribuisce

```
for (int i=0; i<S; i++)
    carte[i] = mazzo_utilizzato → PesaCarta();
```

La valuta crea 2 vettori, uno lungo quanti i valori e uno di semi ← conto così ho

2 tipi di punti → valori uguali o segni uguali

↑ valori di coppia

cerco 1, 2 massimi

4 - 1 ⇒ ho poker se 1° max è 4 non prendo il secondo, negli altri casi si

3 - 2 ⇒ full

3 - 1 ⇒ tris

2 - 2 ⇒ doppia coppia

2 - 1 ⇒ coppia ^{vestita} etc

1 - 1 ⇒ tante possibilità

array si può usare per vettori statici ← ha gli stessi operatori di selezione
↑ no push-back etc...

if annidati o più if personali

valuta → usa solo i vettori delle carte ← non ha lunghezza fissate, dipende dal tipo di
↳ restituisce un punto mazzo

inizia calcolando il primo e secondo max (anche valori uguali)

↳ coppia vestita è data da J o A come valori

per i massimi uso un pair

conta: riempie i due vettori

cerca massimi: parto dal terzo valore e i primi due sono i primi massimi
↳ qui dico entrambi a 0 perché il valore ha valori positivi
↳ allungare al secondo max che va confrontato anche lui

default: if (scala) ← tutte contestezze seme
{ if (colore)
 { // ho scala reale
 if (ho asso e re)
 { // ho scala reale max
 }
 else
 { // scala reale normale
 }
 }
 else
 { vede se ho un 5 nei semi
 { if (colore)
 { colore
 }
 }
 else
 { niente
 }
 }
}

so che è ≠ 1 ma ci sarebbe accoppiamento

cerca il primo non zero ↑ con la f. chiamata
• vedo se il valore è ≠ 1 o se sono arrivati in fondo
• cerco 4 carte a 1 di fila con while
 re ne trovo una ≠ 1 return false
• la scala minima verifica pos asso, 5, 4, 3, 2 (condizione esplicita)

Gli operatori di output sono fondamentali per debug anche se non lo si usa nel programma

Generatori randomici sono pseudo random e vengono generati a partire da un seme

↑ operazioni casuali a partire da X (seme)

ci sono generatori che forniscono il valore iniziale dopo $2^{19937} - 1$ volte ←

in C si usava come seme 'time'

in C++ c'è una libreria che fornisce delle funzioni random ← si usa per generare il seme di questo algoritmo

static mantiene il valore nelle chiamate successive della funzione

(rand() % (b-a+1)) + a ← in C

In cpp

```

random_device rd; ← no seme
mt_19937 gen(rd()); ← algoritmo di generazione
uniform_int_distribution<int> dist(a,b); ← distribuzione uniforme
return dist(gen);

```

Videopoker

Mano e Mazza come oggetti nel videopoker senza uso di pointer ← unico pointer è mano → mazzo per il multiplayer che usano lo stesso mazzo

Funzioni: Gioca(); AggiornaCrediti(); funzioni ausiliari di output

private: f ausiliari della gioca (es. cambia carte etc...)

C. default (credito) ← vettore con i punti per ogni vincita
: vincite { 0, 0, ..., 800, mano (& mazzo) ← singolo pacchetto

```

{ credito = c; }

```

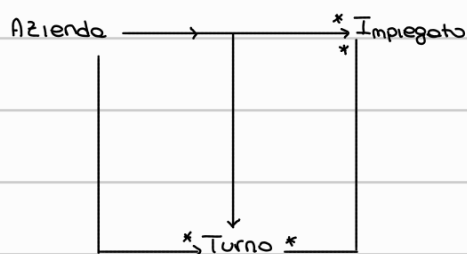
Distribuisce Mano()

```

{
  mazzo.Rimescola();
  mano.Distribuisce();
}

```

Es 6.10 UML esercizio azienda



il turno è legato all'impiegato in una azienda

⇒ tutte le responsabilità sull'azienda ⇒ scelta obbligata perché Impiegato e Turno sono fatti

In azienda c'è un vettore di puntatori a impiegati; vettore di turni; vettore di coppie <impiegato, turno>

↳ canonicamente puntatori

↳ lui propone, per le specifiche (impiegati devono essere presenti nel vett), di inserire gli indici nella coppia.

Funziona per le altre due associazioni di azienda

⇒ forza l'utilizzo di turni e impiegati esistenti dovendo fare

nessariamente i controlli

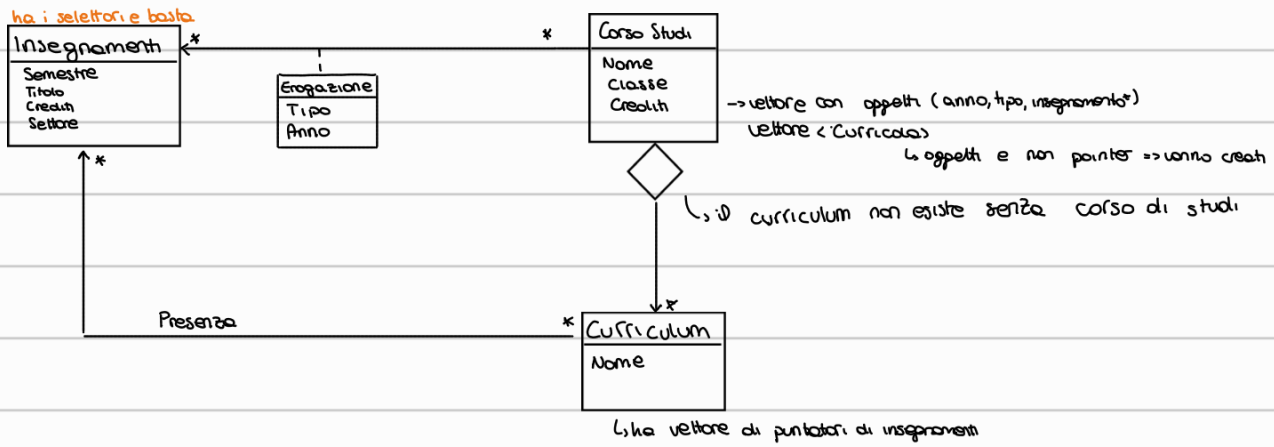
⇒ class/struct esterna public perché interna

Azienda con costruttore (uns. max-impiegati) → Azienda::Azienda(uns. max-impiegati)
: MAX-IMPIEGATO (max-impiegati)

In VediAssegnazione torrio Pair<Impiegato*, Turno*>

Precondizioni: if (condizione da evitare) throw invalid_argument("...");

ES 6.13 Corso di studi



Per l'output implemento l'operatore "<<" in tutte le classi e lo sfrutto per quelle più alte nella gerarchia.

```

Inscrivi Persona (p: Persona) ← push-back (p)

Inscrivi Persona (name: String, cognome: String, ...)
{
    Persona p(name, cognome, ...);
    push-back (&p);
}

Persona* pp = new Persona (name, cognome, ...) ← allocare oggetti anonimi
p.push-back (pp);
corretto, andrebbe creato anche il distruttore
    
```

↳ punta a un oggetto che "muore"

sbagliato: p non esiste fuori dalla funzione

parentesi di spiegazioni

Curriculum

```

string nome;
vector<Insegnamento*> insegnamenti;
unsigned crediti_tot; ← crediti di tutti gli insegnamenti, dato ridondante ma comodo per quando chiede la somma dei crediti
int CercaInsegnamento (Insegnamento* in) const;
    
```

corso di studi, faccio una classe erogazione (Tipo, Anno, Insegnamento*)

enum

public: ...

```

Crea Curr. (string Nome);
InsInInsegn (Ins* in, Tipologia t, uns. a)
InsInCurr (Ins* in, string curr)
    
```

Inscrivi In Ins

```

if (Cerca Ins (in) == -1)
    erogazioni.push_back (Erog. (in, t, a)); ← lo attacco
    
```

pointer

Crea Curr

```

precond.
curricula.push_back (Curr. (nome)); ← lo creo
    
```

Inscrivi In Curr

```

c = Cerca Ins
; = Cerca Curr
↳ precondizioni
curricula[c].InsIns (in);
    
```

consipnet **Somma Crediti** (tipologia +) ← restituisce i crediti della tipologia

f esterna ← vett < CorsoStudi > n Crediti dei corsi ^{obbligatori} in comune a tutti

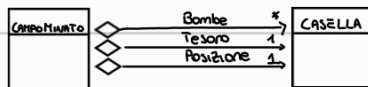
↳ puntatori
Insegnamenti in comune → vettore da CorsoStudi[0] di tipo obbligatori
↳ solo l'insegnamento, non la tripla

faccio il giro di ins. e se lo trovo come non obbligatorio / non lo trovo lo tolgo dal vettore

↓
devo usare un while che incremento l'indice solo se non elimino

mi rimane il vett di tutti gli ins. obbligatori in comune di tipo obbligatorio ← faccio la somma alla fine dei crediti

6.14



soluzione che non richiede l'uso di una matrice ← perché ho la casella